

Supplementary Material

ANONYMOUS AUTHOR(S)

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Software vulnerability detection, Large language models, Retrieval-augmented generation, Hypothesis-guided retrieval, Hypothesis-level verification, Historical vulnerability fixes

This appendix contains detailed experimental configuration; the full 15-family taxonomy; mechanism-level analysis of generated hypotheses; token-cost analysis; and the prompts used throughout the framework. The data, definitions, and experimental settings reported here are consistent with the main-paper version.

A Detailed Experimental Configuration

A.1 Runtime Environment

For Qwen3.8-27B, the model is deployed locally on NVIDIA A800 GPU nodes in an HPC cluster; embedding, retrieval, reranking, LLM inference, and result post-processing are all performed locally. For GLM-4.5 and DeepSeek-V4, LLM inference uses the official APIs, while auxiliary tasks—embedding, retrieval, reranking, and result post-processing—run locally on NVIDIA A40 GPU nodes in the HPC cluster.

A.2 Baseline Reproduction

For Direct LLM, we use a JSON-only prompt template consistent with our task definition. For CodeRAG, we follow the code-level RAG baseline setup in Vul-RAG [4], using sparse BM25 retrieval over historical vulnerable/fixed function pairs from the PrimeVul-v0.1 [3] training set. For LLM4Vuln, we build a vulnerability knowledge base from the PrimeVul-v0.1 training set following its original pipeline, perform dense retrieval using a functional summary of the target function, make separate judgments with the Top-3 knowledge items, and aggregate them by majority vote. Because the original Vul-RAG paper does not evaluate on PrimeVul-v0.1, we re-extract vulnerability knowledge following its original pipeline and build a textual vulnerability knowledge base from the PrimeVul-v0.1 training set.

A.3 Retrieval and Reranking Components

For HypoRAG, we use ChromaDB [2] as the vector database, bge-code-v1 [5] as the dense embedding model, and bge-reranker-v2-m3 [1] as the base cross-encoder reranker. LLM4Vuln also uses ChromaDB and bge-code-v1 for dense retrieval; CodeRAG and Vul-RAG use sparse BM25 retrieval following their official setups.

A.4 Cross-Backbone Configuration

When changing the backbone, we rerun all LLM-dependent offline construction for that backbone. This includes building HypoRAG’s structured historical repair-knowledge base, generating teacher preference labels, and fine-tuning the verification-utility reranker, as well as extracting vulnerability knowledge for LLM4Vuln and Vul-RAG. CodeRAG does not use LLM-driven offline knowledge

extraction, so it reuses the same retrieval corpus across backbone settings; detection still uses the corresponding backbone model.

A.5 Preference-Data Construction and Split

Reranker preference data are constructed only from the official PrimeVul-v0.1 training set. We generate local vulnerability hypotheses and candidate knowledge units from the training samples, and a teacher LLM labels the verification utility of each candidate for the current hypothesis. We then split preference samples into training, validation, and evaluation subsets in an 8:1:1 ratio, grouping by source repair case. This grouping ensures that hypotheses, candidate knowledge units, and preference pairs derived from the same repair case do not cross subset boundaries. The three subsets are used for reranker training, hyperparameter selection, and ranking-quality evaluation, respectively.

A.6 Hyperparameter Settings

All LLMs use deterministic decoding with a maximum generation length of 8,192 tokens. In HypoRAG, each target function generates at most three local vulnerability hypotheses. Each retrieval view retains the top 20 candidate knowledge units; their merged and deduplicated set forms the candidate pool. A cross-encoder reranks this pool, and the top three units are used for hypothesis-level verification. These settings are selected on the validation set; the test set is not used for hyperparameter selection. The reranker is fine-tuned with a pairwise ranking objective using the AdamW [6] optimizer, a learning rate of 2×10^{-5} , weight decay of 0.01, three epochs, an effective batch size of 32, and a warmup ratio of 0.1. The maximum sequence lengths for training and inference are 1,024 and 4,096, respectively, and training uses bfloat16 precision.

B Details of Real-World Vulnerabilities

HypoRAG flags 39 functions as potentially vulnerable in the real-world audit. Manual review confirms 22 genuine security vulnerabilities, while the remaining 17 candidates are false positives. Table 1 provides the complete set of 20 confirmed vulnerabilities that had been fixed upstream by the paper’s cutoff date. The two remaining confirmed vulnerabilities were still unfixed and are therefore omitted from the table to avoid disclosing actionable details before remediation.

Among the 20 fixed vulnerabilities, 14 were independently fixed by the project maintainers in upstream commits after release r2117. These fixes were not triggered by our reports, but the corresponding upstream patches independently address the same security issues identified by HypoRAG. We reported the remaining eight confirmed vulnerabilities to the maintainers; six had been fixed by the cutoff date following our reports and the maintainers’ adoption of our repair recommendations, while two remained unfixed. We mark these six report-triggered fixes with [†] in Table 1.

The confirmed vulnerabilities cover a range of security-relevant failure modes, including stack and heap buffer overflows, out-of-bounds reads and writes, use-after-free, divide-by-zero denial of service, and resource-exhaustion vulnerabilities. Five findings have additionally been assigned CVE identifiers: CVE-2026-86514, CVE-2026-86515, CVE-2026-92879, CVE-2026-92880, and CVE-2026-92881.

C Full Mechanism-Family Taxonomy

The mechanism family is a categorical retrieval constraint rather than a CWE label. Table 2 gives the complete 15-family taxonomy used for historical repair cases and local vulnerability hypotheses. The scope column states the local verification condition that defines each family. CWE labels and

Table 1. Confirmed real-world vulnerabilities identified by HypoRAG and fixed upstream.

Location	Root Cause	Security Impact	Upstream Fix
base/streamfile_stdio.c:fopen_win()	Wrong wide-char destination count.	Stack buffer overflow.	5af3e5d 2026-07-10
coding/libs/relic_lib.c:unpack_frame()	Byte-count used as element-count bound.	Out-of-bounds read.	cb1e203 2026-07-10
meta/txtip_parser.c:txtip_parse()	Unbounded sscanf scansets.	Stack buffer overflow. CVE-2026-86514	4669d37 2026-07-11
meta/txtip_parser.c:add_entry()	Unbounded range expansion with five-entry realloc growth.	Memory and CPU exhaustion DoS. CVE-2026-86515	4b6a02d 2026-07-21
util/crt_utf.c:utf_open()	Unchecked schema/row bounds.	Heap out-of-bounds read.	b7df1e4 2026-07-26
util/m2_psb.c:decode_key()	Non-terminating key chain caused off-by-one write.	Off-by-one stack write.	7eb768e 2026-07-26
base/tags.c:vgmstream_tags_reset()	Unchecked copy into fixed buffers.	Buffer overflow.	67e7822 2026-07-31
meta/acb.c:acb_cat()	strlen(dst) used to validate source length.	Buffer overflow.	538441f 2026-08-02
meta/audiopkg.c:v_strcat()	Faulty destination-based length check.	Buffer overflow.	538441f 2026-08-02
meta/sqex_sead.c:sead_cat()	Faulty destination-based length check.	Buffer overflow.	538441f 2026-08-02
coding/ima_decoder.c:decode_ms_ima()	Non-positive frame geometry used in division/modulo.	Divide-by-zero DoS.	609c0d0 2026-08-11
meta/dec.c:get_falcom_looping()	Unbounded sscanf into stack buffer.	Stack buffer overflow.	e8fc8ce 2026-08-14
meta/ngc_dsp_std.c:init_vgmstream_dsp_apex()	Zero channel count before division.	Divide-by-zero DoS.	e8fc8ce 2026-08-14
coding/ngc_dsp_decoder.c:decode_ngc_dsp()	Unchecked DSP predictor index.	Out-of-bounds read.	52741d9 2026-08-16
coding/psx_decoder.c:ps_find_padding()	Reverse scan read from a negative buffer position.	Stack buffer underflow / out-of-bounds read.	3e66471 [†] 2026-08-20
meta/mus_acm.c:parse_mus()	Unbounded track count used in allocations.	Excessive allocation / DoS. CVE-2026-92879	ae37662 [†] 2026-08-23
coding/vadpcm_decoder.c:vadpcm_read_coefs_be()	Negative entry count bypassed bounds check.	Heap out-of-bounds write. CVE-2026-92880	ae37662 [†] 2026-08-23
meta/awb.c:init_vgmstream_awb_memory()	Zero offset alignment used as modulo divisor.	Divide-by-zero DoS. CVE-2026-92881	ae37662 [†] 2026-08-23
meta/txtip_process.c:make_group_random()	Stale pointer closed after ownership transfer.	Use-after-free.	ae37662 [†] 2026-08-23
base/sbuf.c:sbuf_silence_part()	Unchecked byte range passed to memset.	Heap buffer overflow.	38848ca [†] 2026-08-24

Dates are upstream committer dates. [†] Fixed following our reports and the maintainers' adoption of our repair recommendations. The two confirmed vulnerabilities that remained unfixed at the paper's cutoff date are omitted to avoid disclosing actionable details before remediation.

mechanism families are not one-to-one: the same CWE can arise from different conditions, and one mechanism family can contain cases with different CWE labels.

D Mechanism-Level Analysis

We examine how local vulnerability hypotheses generated by different backbones are distributed across the mechanism space. Figure 1 reports the proportion of hypotheses assigned to each of the 15 mechanism families for the three backbones.

The generated hypotheses span a broad range of mechanism families, indicating that the hypothesis-generation stage does not concentrate on only a small subset of vulnerability patterns. At the same time, their distributions are not uniform across backbones: some mechanism families appear frequently across multiple models, whereas others exhibit stronger model-specific

Table 2. Complete 15-family mechanism taxonomy used for hypothesis generation and retrieval.

Mechanism family	Verification scope
Parser Cursor / Remaining Extent	A parser or protocol cursor consumes input without first confirming that enough bytes remain.
Direct Transfer Extent	An explicit read, write, or copy length exceeds the available source or destination range.
Capacity / Index Extent	An index or traversal range exceeds the valid capacity of the referenced object.
Allocation / Transfer Extent	An allocation or data-transfer size does not cover the required data extent.
Size / Offset Arithmetic	Integer overflow, underflow, or type conversion produces an incorrect size or offset.
Structure / Shape Cardinality	Dimensions, strides, element counts, or related shape/cardinality quantities are inconsistent.
Object Validity / Type / Lookup	A pointer, object, handle, iterator, type, or lookup result is not valid before use.
Concurrency / Ordering / Atomicity	Shared state is accessed with a race, missing synchronization, or incorrect ordering/atomicity.
Information Exposure / Output Control	Sensitive or internal data is exposed through output, logs, memory, or privilege-related behavior.
Cycle / Revisit Termination	Cycles or repeated visits can prevent traversal or processing from terminating.
Depth / Budget Termination	Recursion, nesting, or a resource budget lacks a safe termination bound.
Divide / Modulo Nonzero	A divisor or modulus can be zero or otherwise invalid.
Shift Width / Representability	A shift amount or numeric representation exceeds a valid width or range.
Parsed Value / Domain Validation	A parsed or external value is not checked against its valid domain or protocol state.
Other	Long-tail mechanisms not covered by the explicitly defined families above.

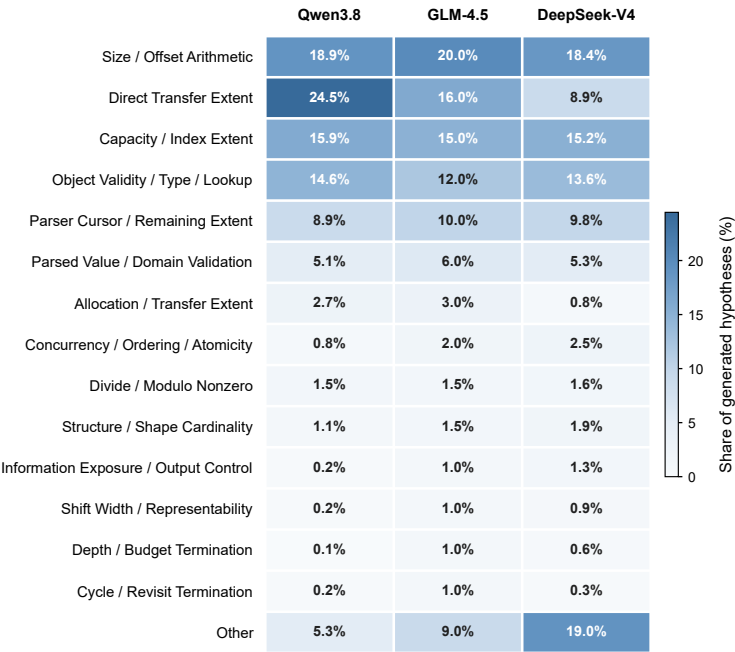


Fig. 1. Distribution of generated vulnerability hypotheses across the 15 mechanism families for different LLM backbones. Each cell reports the percentage of hypotheses assigned to the corresponding family. Mechanism families are ordered by their average frequency across the three backbones, with *Other* shown last.

preferences. Thus, even for the same target functions, different backbones may emphasize different local security conditions and construct different hypothesis-level representations of potential vulnerabilities.

E Token-Cost Experiment

We separately measure the online token cost and the corresponding detection performance to make the efficiency trade-off explicit. Figure 2 reports the average cost per target function for the

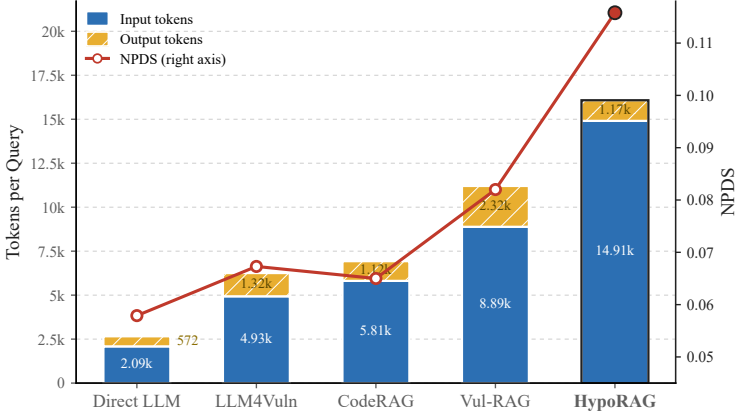


Fig. 2. Token-cost and NPDS comparison on the GLM-4.5 backbone. Bars show average input and output tokens per target function; the line shows NPDS on the right axis.

GLM-4.5 setting; token costs are comparable across the three backbones. The input and output counts are measured before aggregation over the test set, and total tokens are their sum.

HypoRAG incurs approximately 16.1k tokens per query, compared with 2.7k for Direct LLM, 6.9k for CodeRAG, 6.3k for LLM4Vuln, and 11.2k for Vul-RAG. This cost primarily comes from multi-stage reasoning around local vulnerability hypotheses, independent verification of each hypothesis, and the additional context introduced by retrieval and reranking. Relative to Vul-RAG, HypoRAG uses approximately 43% more tokens while improving NPDS by 41.1%–58.0% across the three backbones. Compared with CodeRAG and LLM4Vuln, it uses approximately 132% and 157% more tokens, corresponding to NPDS gains of 74.6%–109.0% and 71.9%–151.4%, respectively. Thus, the detection gains entail a substantial online cost and do not imply higher token efficiency under every comparison. The cost is about six times that of Direct LLM, whose lower cost comes with limited pairwise discrimination (Coverage is only 0.10–0.19).

The cost is concentrated on the input side: HypoRAG uses 14.9k input tokens but only 1.2k output tokens, fewer than Vul-RAG’s 2.3k. The additional cost therefore comes primarily from injected retrieval context rather than lengthy generation. Input-side costs may be amortized through prompt caching, whereas generation-side costs are harder to reduce.

The trade-off is not uniform across backbones. Although HypoRAG outperforms the baselines with Qwen3.8-27B, its NPDS (0.0812) is lower than with the other two backbones. Several stages depend on the backbone: errors in offline knowledge construction, teacher preference labeling, and online hypothesis generation may accumulate downstream. The reranker results are consistent with this explanation: its R@3 is 0.6925 for Qwen3.8-27B, below 0.8109 for GLM-4.5 and 0.7688 for DeepSeek-V4. Thus, the final gains remain constrained by the backbone’s code-understanding and security-reasoning capabilities.

In addition to online costs, HypoRAG incurs one-time offline preparation costs for building the structured repair-knowledge base, generating approximately 60,000 teacher-labeled query–candidate pairs, and fine-tuning the reranker for three epochs (about eight A800 GPU-hours). These expenses can be amortized across subsequent detections.

F Prompt Templates from the Released Implementation

The public artifact provides the executable prompts used by the current online pipeline and offline data-construction stages. This appendix includes only the prompt text: system messages, user-message templates, output schemas, and the frozen taxonomy payload injected into those prompts. Python implementation code and command-line orchestration are intentionally omitted. The prompt text is reproduced from release commit 5611eb5bb0d7 of the repository.

The current implementation uses one frozen 15-family taxonomy. Hypothesis-generation and route-retrieval prompts serialize the complete allowed-family map, and every emitted family label is validated against that map. The point-verification and reranker prompts are intentionally family-neutral: they consume a hypothesis, evidence, or candidate fields and do not reclassify the mechanism. Function-level aggregation is deterministic, so the current release has no separate LLM aggregation prompt.

F.1 Frozen 15-family taxonomy payload

Listing 1. Frozen 15-family taxonomy payload injected into generation and retrieval prompts.

FROZEN TAXONOMY PAYLOAD INJECTED INTO THE PROMPTS Taxonomy key: allocation_state_representation_other_absorbs_lifetime_error_protocol_15 Allowed mechanism families: - PARSER_CURSOR_REMAINING_EXTENT: A parser, decoder, protocol, or file-format cursor/offset is advanced or dereferenced without proving that the required bytes remain between the current position and a visible end boundary. This includes a pointer/offset whose associated remaining-length value is not updated consistently. - DIRECT_TRANSFER_EXTENT: A read, write, copy, initialization, or fixed-buffer formatting operation uses an amount that can exceed a non-allocated source or destination extent. - CAPACITY_INDEX_EXTENT: An array, table, string, collection, or fixed-capacity object is indexed, iterated, or copied beyond its declared element count or capacity. - ALLOCATION_TRANSFER_EXTENT: A dynamically allocated or resized object has an incorrect capacity/ownership contract for a later transfer even though no arithmetic, fixed-buffer bound, or multi-dimensional layout relation is the earlier failed condition. - SIZE_OFFSET_ARITHMETIC: Overflow, underflow, wraparound, or signed/unsigned conversion first creates a wrong size, count, offset, allocation extent, or array index. Use this when a negative or narrowed value becomes an extent/index rather than when it only violates an operation-specific domain. - STRUCTURE_SHAPE_CARDINALITY: Relations among dimensions, shapes, parallel collections, record counts, split/stride tables, or mode-dependent layout, including whether a width/height/depth/header relation can be represented by the allocated layout. - OBJECT_VALIDITY_TYPE_LOOKUP: Object absence, invalid type/tag, failed lookup, invalid iterator/end state, optional presence, or invalid helper result. - CONCURRENCY_ORDERING_ATOMICITY: Missing synchronization, interleaving, atomicity, RCU, or publication/order across threads. - INFORMATION_EXPOSURE_OUTPUT_CONTROL: Outward stale/sensitive data, logging, timing, encoding, or response control. - OTHER: No more-specific supported fit. This includes ownership transfer, release, refcount, invalid reference, cleanup order, callback-invalidated aliases; unchecked failures, initialization, error propagation, or rollback failures; and remaining protocol/application sequence, authorization, identity/capability, configuration, or trust-boundary invariants. Do not use OTHER when a remaining specific family directly describes the earliest failed safety condition. - CYCLE_REVISIT_TERMINATION: Recursive or graph traversal can revisit a previously visited node, pixel, object, pointer target, or state because a visited/mark/cycle invariant is absent or not propagated to the recursive call. Use only when re-entry/cycle, rather than depth alone, is the direct failure. - DEPTH_BUDGET_TERMINATION: Recursion or repeated processing can exceed an explicit maximum depth, iteration budget, or resource budget because the threshold check is absent, off by one, or occurs too late. Use only when a depth/budget bound, rather than revisitation, is the direct failure. - DIVIDE_MODULO_NONZERO: A division, modulus, reciprocal, or ratio operation is reached without proving its divisor/denominator is nonzero and otherwise valid. The immediate failed condition is the denominator precondition. - SHIFT_WIDTH_REPRESENTABILITY: A shift or bit-position operation is reached without proving that the shift count is within the operand width and that the shifted result remains representable. The immediate failed condition is shift-count/representability, not general arithmetic overflow.

- PARSED_VALUE_DOMAIN_VALIDATION: A parsed, decoded, configured, or externally supplied scalar is used as a mode, enum, dimension class, limit, or other semantic parameter without proving it belongs to the required allowed domain. Do not use for direct arithmetic overflow, divide/modulus, or shift preconditions.

Precedence:
Apply allocation and state rules first. For numeric semantics, use SIZE_OFFSET_ARITHMETIC when conversion/sign creates a wrong extent/index, DIVIDE_MODULO_NONZERO for denominator preconditions, SHIFT_WIDTH_REPRESENTABILITY for shift preconditions, and PARSED_VALUE_DOMAIN_VALIDATION for the remaining semantic range/enum faults. Use OTHER for the removed lifetime, error, and protocol/trust conditions unless a remaining specific family is the earlier failed condition.

F.2 Knowledge-base construction prompts

Listing 2. Repair-signature extraction prompt text.

PROMPT: REPAIR-SIGNATURE EXTRACTION (system message)

Extract one primary Mechanism Signature from a vulnerability repair pair. The signature represents the earliest security-relevant failed safety condition repaired by the actual code change. The vulnerable and fixed functions, their diff, and a CVE description are available. The code change is primary evidence; metadata may be incomplete or misleading.

Use neutral technical language. Do NOT name a CWE, vulnerability category, mechanism family, taxonomy label, or broad subtype. Do not infer helper or caller contracts that are not visible. Do not turn tests, formatting, renames, or unrelated changes into a vulnerability. When the change is unclear, state the uncertainty.

The before and after excerpts must be short, exact, nonempty substrings of the respective functions. Return JSON only.

Schema:

```
{
  "adjudicability": "clear|uncertain|unrelated_patch",
  "operation": "dangerous operation in reusable semantic language",
  "risk_object": "object acted upon",
  "trigger_condition": "specific bad-state relation",
  "violated_invariant": "safety relation required before the operation",
  "failed_protection": "guard, ownership, state, or error action that failed",
  "repair_action": "what the patch changes to establish the invariant",
  "evidence_before": "short exact vulnerable source excerpt",
  "evidence_after": "short exact fixed source excerpt",
  "uncertainty": "none or the missing context",
  "reason": "one diff-grounded sentence"
}
```

PROMPT: REPAIR-SIGNATURE EXTRACTION (user message schema)

The user message is a JSON object with these fields:

```
{
  "vulnerable_function": "<vulnerable function source>",
  "fixed_function": "<fixed function source>",
  "repair_diff": "<unified diff between the two functions>",
  "cve_description": "<CVE description or empty string>"
}
```

Listing 3. Taxonomy-routed M/R/E retrieval prompt text.

PROMPT: TAXONOMY-ROUTED M/R/E RETRIEVAL

SYSTEM MESSAGE -- shared route style

You create one concise retrieval representation from a frozen, taxonomy-neutral security description. Jointly choose the primary mechanism family and write the requested route field in the SAME JSON response. Do not first assign a family and then mechanically restate its definition.

The supplied description is the only evidence. Do not infer missing source, caller, helper, patch, label, CWE, CVE, project, or API facts. Follow the given

taxonomy definitions and precedence exactly. Choose OTHER only when no remaining specific family describes the earliest failed safety condition. Style contract shared by every route: write one factual, reusable sentence of at most 55 words. Use the vocabulary operation, object, trigger/invariant, guard, or repair action when supported. Preserve important distinctions such as cursor remaining extent versus fixed capacity, arithmetic-created extent versus direct transfer, and semantic scalar domain versus object/type validity. Avoid generic phrases such as 'validate input' or 'prevent vulnerability', broad CWE labels, and project-specific narrative. Return JSON only. SYSTEM MESSAGE -- injected taxonomy and route objective Frozen taxonomy: <pre>{ "allowed_families": "<the complete 15-family map>", "precedence": "<taxonomy precedence>" }</pre> Route objective: <one of the following> - MECHANISM route: State the alleged failed safety mechanism: the concrete operation, risk object, and trigger/invariant relation that would need checking. Do not prescribe a repair. - REPAIR route: State the specific validation, guard, ownership action, or invariant-establishing repair principle that a useful historical case should provide. - EVIDENCE route: State the local evidence or relation that must be checked to confirm or reject this hypothesis: operation, relevant values/state, and required guard. Schema: <pre>{ "mechanism_family": "one allowed family", "<route field>": "one concise sentence" }</pre> USER MESSAGE <pre>{ "signature_id": "<id>", "source_kind": "hypothesis or knowledge", "neutral_description": "<taxonomy-neutral description>" }</pre> SYSTEM MESSAGE -- joint M/R/E variant Use the same shared route style, frozen taxonomy, and route objectives above. Return one shared mechanism_family for all three route fields. The family must describe the earliest failed safety condition, not merely the wording preferred by one route. Each field must answer its own route objective. Do not make M, R, and E three near-duplicate sentences or copy a family definition into all fields. Joint schema: <pre>{ "mechanism_family": "one allowed family", "mechanism_claim or mechanism_observed": "one concise sentence", "repair_sought or repair_applied": "one concise sentence", "evidence_to_check or evidence_decisive": "one concise sentence" }</pre>

Listing 4. Guidance-package prompt for generating the four verification fields from a neutral repair signature.

```
PROMPT: KNOWLEDGE-BASE VERIFICATION GUIDANCE PACKAGE

SYSTEM MESSAGE

You create verification guidance from ONE neutralized historical repair signature.

A later verifier will see only the four requested fields, a current function, and a suspicious hypothesis. The
verifier will not see the original repair code, diff, project name, CVE, or the complete historical case.
Therefore make every field concrete and self-contained.

Use only relations present in the supplied neutral signature. Do not invent API contracts, exploitability, patch
details, or facts absent from the signature. These fields are mechanism hints, never proof that another
function is vulnerable.

Field requirements:
- case_explanation: State the historical operation, risk object, failed safety relation, consequence, and why its
  repair mattered.
- vulnerable_pattern: State the pre-repair unsafe relation, the trigger, and the consequential operation.
- repaired_pattern: State the guard, state change, or invariant-establishing repair and the unsafe relation it
  prevents.
- verification_guidance: State actionable checks in another function: the operation, values or state, safety
  relation, and local guard that could confirm or reject a similar concern.

Before producing the final JSON, reason thoroughly and privately about the supplied signature. Keep private
reasoning out of the final output. Each field may be up to 120 words. Return JSON only, with exactly this
schema:
{
  "case_explanation": "one self-contained concise sentence or short paragraph",
  "vulnerable_pattern": "one self-contained concise sentence or short paragraph",
  "repaired_pattern": "one self-contained concise sentence or short paragraph",
  "verification_guidance": "one self-contained concise sentence or short paragraph"
}

USER MESSAGE

{
  "candidate_signature_id": "<historical repair signature ID>",
  "neutral_repair_signature": {
    "adjudicability": "<clear|uncertain|unrelated_patch>",
    "operation": "<dangerous operation>",
    "risk_object": "<object acted upon>",
    "trigger_condition": "<bad-state relation>",
    "violated_invariant": "<required safety relation>",
    "failed_protection": "<failed guard or action>",
    "repair_action": "<invariant-establishing change>",
    "evidence_before": "<vulnerable source excerpt>",
    "evidence_after": "<fixed source excerpt>",
    "uncertainty": "<none or missing context>",
    "reason": "<diff-grounded explanation>"
  }
}
```

F.3 Joint hypothesis-generation prompts

Listing 5. Joint hypothesis-generation prompt text.

```
PROMPT: JOINT HYPOTHESIS GENERATION

SYSTEM MESSAGE -- task instruction

Analyze the given function and identify zero to three concrete potentially dangerous operation points as security
hypotheses. A hypothesis does not need to prove that a vulnerability definitely exists. It must identify a
concrete, locally plausible failed safety condition worth subsequent retrieval and verification.
```

Before selecting final hypotheses, privately scan the whole function for security-relevant operations and state transitions that could become unsafe under locally plausible conditions. Consider memory access or transfer, size and offset arithmetic, parsing and cursor movement, indexing, allocation, lifetime or ownership transitions, representation conversion, state or permission checks, and error-sensitive operations when present. Then select up to three distinct candidates that provide the strongest coverage of plausible failure points.

Every final hypothesis must identify a concrete operation, risk object, specific trigger condition, safety invariant required before that operation, and a short exact code anchor copied from the function. Ground a hypothesis in a concrete operation plus a plausible unsafe relation among visible values, objects, states, or control flow; do not propose one solely because a generic validation check is absent. Treat unseen caller or helper behavior as uncertainty, not as proof that a candidate is safe or unsafe.

Do not discard a candidate merely because a check, guard, bound, clamp, reset, or error path exists nearby. Determine whether it establishes the required invariant for the same values, object, execution path, and operation. Discard a candidate only when the visible code clearly establishes that invariant at the operation. If the protection may be incomplete or its sufficiency cannot be determined locally, retain the candidate and state the uncertainty. A debug assert alone is neither proof of a defect nor a production guarantee.

When multiple slots are used, maximize coverage of distinct dangerous operations or distinct failed invariants. Do not spend separate slots on branch-by-branch or near-duplicate variants of the same concern. Do not stay conservative by collapsing distinct, locally plausible candidates into one.

First identify each candidate failure condition from the code itself. Only then assign exactly one primary mechanism_family using the frozen taxonomy definitions and precedence; taxonomy assignment must not suppress, merge, or reshape an otherwise locally plausible hypothesis. Choose OTHER only when no remaining specific family fits. Finally write M/R/E for the selected hypothesis.

SYSTEM MESSAGE -- shared M/R/E style

Style contract shared by every route: write one factual, reusable sentence of at most 55 words. Use the vocabulary operation, object, trigger/invariant, guard, or repair action when supported. Preserve important distinctions such as cursor remaining extent versus fixed capacity, arithmetic-created extent versus direct transfer, and semantic scalar domain versus object/type validity. Avoid generic phrases such as 'validate input' or 'prevent vulnerability', broad CWE labels, and project-specific narrative. Return JSON only.

SYSTEM MESSAGE -- private analysis profile: focused

Before producing the final JSON, privately make one complete candidate-discovery pass across the function. Identify distinct security-relevant operations, their risk objects, locally plausible triggers, and required invariants. Check whether visible protections clearly establish each invariant for the same values, object, execution path, and operation; retain a distinct candidate when that sufficiency remains uncertain. Do not retain candidates lacking a concrete locally plausible unsafe relation, but do not discard distinct candidates merely because a nearby protection exists. Select up to three candidates that maximize coverage, then assign taxonomy families and formulate M/R/E. Keep this analysis private and emit the complete JSON object only.

SYSTEM MESSAGE -- private analysis profile: thorough

Before producing the final JSON, first perform a full-function candidate-discovery pass. Trace control-flow reachability and identify security-relevant operations, risk objects, trigger conditions, and required invariants. For each candidate, determine whether visible protections clearly establish the required invariant for the same values, object, execution path, and operation. Do not discard a candidate merely because a protection exists; preserve locally plausible candidates when protection sufficiency remains uncertain. Then select up to three distinct candidates that maximize coverage of plausible failure points, assign taxonomy families, and formulate M/R/E. Do not finish early for apparently simple code. Keep that analysis in the thinking channel, then emit the complete JSON object only.

SYSTEM MESSAGE -- injected taxonomy and output schema

Frozen taxonomy:

```
{ "allowed_families": <the complete 15-family map>, "precedence": "<taxonomy precedence>" }
```

Route objectives: mechanism_claim, repair_sought, and evidence_to_check, using the route definitions above.

Schema:

```
{ "hypotheses": [ { "id": 1, "code_anchor": "short exact nonempty source excerpt", "operation": "dangerous
operation in reusable semantic language", "risk_object": "object acted upon", "trigger_condition": "
specific bad-state relation", "violated_invariant": "safety relation required before the operation", "
uncertainty": "none or the missing local fact", "mechanism_family": "one allowed family", "mechanism_claim
": "one concise sentence", "repair_sought": "one concise sentence", "evidence_to_check": "one concise
sentence" } ] }

USER MESSAGE
{ "idx": "<sample index>", "function": "<target function source>" }
```

F.4 Reranker preference-labeling prompts

Listing 6. Reranker preference-labeling prompt text.

PROMPT: RERANKER PREFERENCE LABELING (system message)

You label verification-utility data for a vulnerability-case reranker.

The downstream reranker must rank historical repair cases by how useful they would be for verifying one repair-oriented suspicious point. Judge reference utility only. Do not decide whether the target function is vulnerable.

You receive exactly three query fields and three candidate fields. Use no other information and do not claim knowledge of source code, project identity, sample provenance, vulnerability labels, patch diffs, or hidden contracts.

Utility labels:

- 2: highly useful. The candidate closely matches the operative risk object, dangerous operation, failure trigger, and protection or repair shape. It provides a discriminating comparison that would materially improve verification.
- 1: partially useful. The mechanism is meaningfully related, but an important risk object, access/copy form, trigger, protection, consequence, or repair detail differs. Examples that share only the weakness class, such as two different NULL dereferences or two different bounds failures, normally belong here rather than label 2.
- 0: not useful. The match is lexical, generic, coarse-family-only, mechanism-mismatched, or unlikely to improve verification.

Direction tags:

- confirm: the candidate's vulnerable mechanism primarily matches the risky path hypothesized by the query.
- rule_out: the candidate primarily supplies a contrasting trigger, protection, or contract condition that would reject the query if that discriminating condition is absent or already satisfied.
- both: the candidate contains two distinct, concrete transferable lessons, one confirmatory and one ruling-out. Do not use both merely because any example could be used to check whether a guard exists.
- none: not meaningfully useful.

Calibration rules:

- A rule_out reference can receive label 2.
- Do not assume that a candidate's guard or repair is present in the target. It is useful only as a comparison that tells the verifier what to check.
- Direction describes the candidate's primary transferable lesson, not the two hypothetical outcomes of inspecting the target. A matching missing-check repair is normally confirm, not both.
- rule_out still requires close alignment with the query's operative mechanism. Use it only when the candidate exposes a concrete discriminating trigger, guard, invariant, contract, or opposite branch for the same kind of risky operation.
- A different vulnerability mechanism or an inapplicable repair is not a rule_out reference. If it merely shows that another bug needs another fix, assign label 0 and direction none.
- A solution field does not prove that the query needs the same repair.
- Shared terminology or mechanism_family-level similarity alone is not enough for label 2.
- For label 2, require close alignment in the concrete operation and trigger,

not just the same broad consequence. Array indexing, loop termination, pointer arithmetic, and length-bounded copying are different shapes even when all can cause out-of-bounds access.

- Before assigning label 2, verify close alignment on all four dimensions: (1) risk object or pointer/value source, (2) dangerous operation, (3) failing trigger or precondition, and (4) protection or repair shape. If one dimension materially differs, use label 1.
- A function parameter, structure field, lookup result, allocation result, and optional flag-controlled pointer are different pointer sources. Their shared need for a NULL check normally supports label 1, not label 2, unless the remaining trigger and contract are also closely aligned.
- Use label 1 when the core mechanism transfers but decisive details differ.
- Label 1 must still contribute at least one concrete transferable check for the same operative mechanism. A broad theme such as "validate before use," "handle errors," or "check bounds" without a comparable operation is label 0.
- Use label 0 confidently for generic advice or a different operative failure.
- label 0 must use direction none.
- labels 1 and 2 must use confirm, rule_out, or both; none is reserved for 0.
- Keep the reason short and ground it only in the six visible fields.
- Describe what the candidate would help the verifier check. Never state that the target vulnerability is confirmed, that the repair is definitely needed, or that an unseen guard is present or absent.

Return one JSON object only, with no Markdown or additional text:

```
{
  "reference_value_label": 0,
  "helpfulness_type": "confirm | rule_out | both | none",
  "reason": ""
}
```

PROMPT: RERANKER PREFERENCE LABELING (user message template)

[QUERY]
retrieval_summary: {retrieval_summary}
needed_example: {needed_example}
evidence_hint: {evidence_hint}

[CANDIDATE]
mechanism_summary: {mechanism_summary}
solution_summary: {solution_summary}
evidence_summary: {evidence_summary}

[TASK]
Judge how useful the candidate is as a comparison for verifying the query.
Return only the required JSON object.

F.5 Point-level verification prompts

Listing 7. Point-level verification prompt text and dynamic user-message structure.

PROMPT: POINT-LEVEL HYPOTHESIS VERIFICATION

SYSTEM MESSAGE -- base prompt

You verify exactly one suspicious vulnerability point against the current code.

Use retrieved historical examples only as weak mechanism hints. They are never proof that the current code is vulnerable.

Return JSON only. Do not include analysis text, Markdown, code fences, or a [FINAL_JSON] marker.

Decision labels:

- supported: the current code itself establishes a concrete, security-relevant, mechanism-complete vulnerability path for the exact suspicious point.
- unsupported: the risky operation is absent, the trigger is not established, visible protection/invariants block the mechanism, or the point is generic/off-target.
- insufficient_evidence: caller contract, helper/API contract, framework invariant, macro behavior, allocator behavior, or external context is decisive but not visible from this function.

Strict supported gate:

- You may choose supported only when all of these are true:
 1. risky_operation_present is yes
 2. trigger_condition_present is yes
 3. protection_status is missing or insufficient
 4. support_evidence contains concrete current-function facts, not historical examples
 5. support_evidence identifies a security impact, not only incorrect behavior, missing style checks, or possible robustness issues
- Do not choose supported for a claim whose main basis is "no explicit check", "could be", "caller might pass bad input", "helper might fail", "debug assert is compiled out", generic OOM behavior, or a historical analogy.

Caller contract and debug assertion discipline:

- A caller-supplied argument being theoretically invalid is not a trigger. If the bad state requires a caller to violate an unstated precondition, choose insufficient_evidence unless the current function visibly accepts untrusted/external input and owns validation.
- Do not infer nullability, size validity, shape validity, ownership transfer, or ordering obligations merely from the absence of a local check.
- A debug-only assertion is evidence of an intended invariant, not proof of a reachable production vulnerability. If the release-build trigger depends on unknown caller behavior, choose insufficient_evidence.
- Checking one parameter for NULL does not prove every other pointer parameter may be NULL.

Helper/API/framework discipline:

- Do not assume helper/API/framework/container/macro semantics from names alone.
- If a framework normally constructs an object, fills builtin fields, validates tensor shapes, owns memory, sends protocol alerts, or establishes map/key invariants, and that contract is not visible here, choose insufficient_evidence rather than supported.
- If visible usage suggests a checked builder, guarded accessor, bounded type, status gate, ownership discipline, or range restriction, choose unsupported unless the current code shows the remaining failing path.

Security-impact discipline:

- For resource leaks, require attacker-triggered repeated accumulation, lost security-sensitive resource, or a visible denial-of-service path. A one-shot or bounded leak is insufficient_evidence or unsupported.
- For allocation failure/OOM claims, require a concrete unsafe dereference, use-after-free, uninitialized output, or security-relevant protocol state failure in the current function. Generic OOM failure handling is insufficient.
- For protocol/error-handling claims, missing a log, alert, or cleanup style is not enough; the current function must show a concrete externally exploitable state or security impact.
- For semantic mismatch or duplicate-input claims, require a sensitive authorization, validation bypass, memory-safety effect, or externally exploitable state. Incorrect comparison alone is not enough.
- For output initialization, supported is allowed only when the current function visibly returns success while leaving a caller-visible output unset or stale.

Off-target discipline:

- Treat the priority field of the suspicious point only as a hint. It cannot substitute for current-code proof.
- If the suspicious point is plausible but unrelated to the evidence_hint, too broad for the visible code, or only a different possible issue, choose unsupported or insufficient_evidence.
- Multiple weak/generic clues must not be combined into supported.

Mechanism-specific discipline:

- Bounds/size/shape: require a specific size/index/count/offset/dimension flowing into allocation, copy, parser read/write, or memory access on a concrete path. Visible range limits, widening, checked builders, or capacity checks count as protection.
- Pointer/lookup: require a specific dereference or lookup result use before a relevant validity check and a visible reason the pointer/key can be invalid here. Do not infer invalid lookup semantics from API names alone.
- Error handling: require a visible success/failure path where output, cleanup, or propagation is wrong and security-relevant.
- Lifetime/concurrency: require a visible free/unref/delete/escaped alias/callback/refcount/race-prone transition on the same path.
- Input/state/permission: require a concrete accepted malformed/unauthorized state reaching a sensitive operation.
- Information exposure: require a concrete outward copy, return, log, permission, or readable data path.

Evidence writing rules:

- support_evidence must be short paraphrases of current-code facts supporting the judgment.
- blocking_evidence must be short paraphrases of guards, checks, validation, cleanup, synchronization, invariants, helper/API usage semantics, ownership discipline, range restrictions, or equivalent current-code facts that block the mechanism.
- Avoid raw code excerpts with backslashes or complex string escapes; paraphrase instead.

Exactly one evidence field per verdict:

- supported: fill support_evidence. Leave blocking_evidence and missing_context empty.
- unsupported: fill blocking_evidence. Leave support_evidence and missing_context empty.
- insufficient_evidence: fill missing_context, and leave support_evidence and blocking_evidence empty.
- missing_context must name the decisive information that is not visible in this function, such as a caller contract, a helper/API contract, a framework invariant, macro behavior, or allocator behavior. Do not restate the suspicious point, and do not speculate about what that information would turn out to be.

Retrieved-example attribution:

- matched_example_ranks must list the "Example rank" numbers of the retrieved case analyses that actually informed this judgment.
- Include a rank only if that case supplied the mechanism framing, the protection pattern, or the check you looked for. Do not list every retrieved example by default.
- Use an empty list when no retrieved example influenced the judgment. This is a legitimate outcome and must not be padded.

Confidence calibration:

- high requires the risky operation, trigger condition, security impact, and missing/insufficient protection to be directly visible in the current function.
- medium may use limited inference from visible current-code facts.
- low means the judgment depends on uncertain caller/helper/API behavior or external context.

Return exactly this JSON shape:

```
{
  "verdict": "supported | unsupported | insufficient_evidence",
  "confidence": "high | medium | low",
  "risky_operation_present": "yes | no | unknown",
  "trigger_condition_present": "yes | no | unknown",
  "protection_status": "missing | insufficient | present | unknown",
  "support_evidence": [],
  "blocking_evidence": [],
  "missing_context": "",
  "matched_example_ranks": []
}
```

SYSTEM MESSAGE -- CASE_MAPPED_SYSTEM_PROMPT (append this suffix to the base prompt)

Case-to-code mapping requirement:

Before choosing the verdict, process every retrieved example separately. For each example, extract one concrete safety principle from its repaired pattern or verification guidance, then map that principle to the current function. The mapping must name the relevant current-code operation, variable, guard, or missing fact. A generic statement that the two cases share a family is not a mapping.

The historical example remains only a source of an inspection principle. It must never substitute for current-code evidence, establish a trigger that is not visible, or by itself justify `supported`.

For every example rank supplied by the user, return one object in

- ```
`case_mappings`:
- `example_rank`: the supplied rank.
- `case_principle`: the specific check, invariant, or protection pattern from that example.
- `current_code_mapping`: how that principle maps to the suspicious operation in the current function, using concrete current-code facts where possible.
- `mapping_status`: exactly one of `established`, `blocked`, `missing_context`, or `not_applicable`.
- `influence`: exactly one of `supports`, `refutes`, `context_only`, or `none`. `supports` means the current code establishes the analogous failing relation; it does not mean the historical case proves the verdict.
```

After all mappings, choose the verdict using the existing strict supported gate. The final support/blocking/missing-context evidence must still be about the current function.

Return exactly this JSON shape:

```
{
 "verdict": "supported | unsupported | insufficient_evidence",
 "confidence": "high | medium | low",
 "risky_operation_present": "yes | no | unknown",
```

```

"trigger_condition_present": "yes | no | unknown",
"protection_status": "missing | insufficient | present | unknown",
"support_evidence": [],
"blocking_evidence": [],
"missing_context": "",
"matched_example_ranks": [],
"case_mappings": [
 {
 "example_rank": 1,
 "case_principle": "",
 "current_code_mapping": "",
 "mapping_status": "established | blocked | missing_context | not_applicable",
 "influence": "supports | refutes | context_only | none"
 }
]
}

```

SYSTEM MESSAGE -- CASE\_MAPPED\_VISIBLE\_EVIDENCE\_SYSTEM\_PROMPT (append this suffix to the base prompt)

Visible-evidence closure:

Do not select `insufficient\_evidence` merely because some caller, helper, or environmental information is absent. Before using that verdict, identify the specific missing fact and ask whether it is genuinely necessary to establish the claimed failure.

Treat the following as decidable from the current function when they are explicitly shown: data/control flow, literal bounds, arithmetic on visible operands, signedness or width stated by the code, fixed local-buffer extent, and a visible guard that does or does not cover the stated trigger. If those facts establish the complete failing relation and security impact, choose `supported` even when unrelated external implementation details are absent.

Conversely, retain `insufficient\_evidence` when the alleged failure depends on an unseen helper's bounds/ownership behavior, an unseen container or structure capacity, a caller-only reachability precondition, privileged environment permissions, or an external protocol contract. Do not invent those facts from a retrieved example.

For each case mapping, make `missing\_context` concrete: name the exact missing fact and explain why the visible current-code relation cannot establish the failure without it. A generic statement that a caller or helper is unseen is not sufficient.

SYSTEM MESSAGE -- CASE\_MAPPED\_VISIBLE\_CARDINALITY\_SYSTEM\_PROMPT (append this suffix to the base prompt)

Concrete extent and cardinality rules:

Treat an operation's requested extent as directly established when the current statement visibly passes both (1) the address of a fixed local destination and (2) an explicit byte/element count whose visible arithmetic can wrap, underflow, or exceed that destination. Do not require the implementation of an ordinary read/write primitive merely to recognize this requested out-of-bounds extent, unless the current code itself shows that the primitive clamps or rejects it.

Likewise, a direct array access may be supported without seeing the allocation site when the current function visibly shows all of the following: an explicit field giving that array's valid entry count, an index bounded by an independent larger cardinality, and no local relation proving the index is below the entry count. In that specific situation, do not call the missing allocation layout decisive. The conclusion must still name the two competing cardinalities and the concrete unchecked access.

These are narrow local-evidence exceptions. They do not apply when the claimed failure instead depends on an unseen helper's semantics, an unspecified container capacity, a caller-only precondition, or an unshown protocol rule.

SYSTEM MESSAGE -- CASE\_MAPPED\_LOCAL\_ARITHMETIC\_SYSTEM\_PROMPT (append this suffix to the base prompt)

Local arithmetic and representation closure:

Treat an arithmetic or representation failure as decidable from the current function when all of the following are visible: (1) a length, offset, index, or count has a wider or signed representation, or is directly subtracted from

or added to another visible value; (2) no visible local guard excludes the bad numeric range; and (3) the resulting narrowed, wrapped, or underflowed value immediately controls a local allocation size, copy extent, array subscript, or pointer offset. In this narrow case, do not require an unseen caller to provide a concrete runtime value merely to recognize the failing numeric relation. State the exact numeric condition and the concrete unsafe consumer in `support_evidence`.

This exception is limited to visible C/C++ arithmetic and representation semantics. It does not permit 'supported' when the decisive fact is an unseen helper or container contract, allocation extent, parser behavior, framework invariant, privilege state, or API return domain. It also does not apply to a division, loop, error path, or abstract state concern unless the visible code already establishes a concrete memory-safety or security-impacting operation.

USER MESSAGE -- dynamically assembled template

[Current Sample]  
Project: <project>  
File: <file>

[Current Code]  
```c  
<target function source>
```

[Suspicious Point To Verify]  
Suspicious summary: <retrieval\_summary>  
Evidence hint: <evidence\_hint>  
Needed historical example: <needed\_example>  
priority: <optional priority>

[Retrieved Historical Case Analyses]  
Example rank: <rank>  
Project: <historical project>  
File: <historical file>  
[Case explanation]  
<historical case explanation>  
[Minimal vulnerable pattern]  
<historical vulnerable pattern>  
[Minimal fixed pattern]  
<historical fixed pattern>  
[Guidance for similar suspicious points]  
<historical verification guidance>

[Task]  
Verify only the suspicious point above. Use retrieved case analyses only as analogies to understand the mechanism and possible protections. Do not treat retrieved examples as proof that the current code is vulnerable. Return only the JSON object required by the system prompt.

## References

- [1] Beijing Academy of Artificial Intelligence. 2024. bge-reranker-v2-m3. <https://huggingface.co/BAAI/bge-reranker-v2-m3>. Hugging Face model card, accessed 2026-08-12.
- [2] Chroma Core. 2026. *Chroma: Open-Source Search Infrastructure for AI*. <https://github.com/chroma-core/chroma>
- [3] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2025. Vulnerability Detection with Code Language Models: How Far Are We?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1729–1741. doi:10.1109/ICSE55347.2025.00038
- [4] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, et al. 2024. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag. *ACM Transactions on Software Engineering and Methodology* (2024).
- [5] Chaofan Li, Jianlyu Chen, Yingxia Shao, Defu Lian, and Zheng Liu. 2025. Towards a generalist code embedding model based on massive data synthesis. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, Vol. 38.
- [6] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.